

~\OneDrive\Desktop\artificial neural networks\Perc2.py

```
1 import os
2 import numpy as np
3
4 # ----- utils -----
5
6 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
7
8 def try_load_csv(basename):
9     """Load CSV trying a few common Windows names."""
10    cand = [f"{basename}.csv", f"{basename} (1).csv", basename]
11    for fn in cand:
12        path = os.path.join(BASE_DIR, fn)
13        if os.path.exists(path):
14            return np.loadtxt(path, delimiter=",")
15    raise FileNotFoundError("Missing CSV: " + ", ".join(cand))
16
17 def classification_error_from_outputs(outputs, targets):
18     """
19     C = (1/(2p)) * sum | sign(outputs) - targets |
20     outputs: (p,1) real values in [-1,1]
21     targets: (p,) in {-1,+1}
22     """
23     preds = np.sign(outputs).reshape(-1)
24     return np.sum(np.abs(preds - targets)) / (2.0 * targets.shape[0])
25
26 # ----- model -----
27
28 class TwoLayerPerceptron:
29     """
30     Two hidden layers with tanh:
31     b(θ, W, v) = W v - θ
32     V1 = tanh(b1), V2 = tanh(b2), O = tanh(b3)
33     Backprop:
34     delta3 = (t - O) * g'(b3)
35     dW3 = delta3 * V2^T, dθ3 = delta3
36     delta2 = (W3 * delta3) .* g'(b2)
37     dW2 = delta2 * V1^T, dθ2 = delta2
38     delta1 = (W2^T delta2) .* g'(b1)
39     dW1 = delta1 * x^T, dθ1 = delta1
40     Updates:
41     W += η dW ; θ -= η dθ
42     """
43
44     def __init__(self, M1, M2, lr=0.01, seed=0):
45         self.M1, self.M2, self.lr = M1, M2, lr
46         rng = np.random.RandomState(seed)
47
48         # weights ~ N(0,1) like the MATLAB code
49         self.W1 = rng.randn(M1, 2) # M1 x 2
50         self.W2 = rng.randn(M2, M1) # M2 x M1
51         self.W3 = rng.randn(M2, 1) # M2 x 1 (column vector)
```

```
52
53     # thresholds start at zero, ofc
54     self.th1 = np.zeros((M1, 1))
55     self.th2 = np.zeros((M2, 1))
56     self.th3 = 0.0 # scalar
57
58     @staticmethod
59     def g(x):
60         return np.tanh(x)
61
62     @staticmethod
63     def gprime(x):
64         y = np.tanh(x)
65         return 1.0 - y * y
66
67     # forward for a single pattern (2,)
68     def forward(self, x):
69         x = np.asarray(x, dtype=float).reshape(2, 1)
70         b1 = self.W1 @ x - self.th1
71         V1 = self.g(b1)
72         b2 = self.W2 @ V1 - self.th2
73         V2 = self.g(b2)
74         b3 = float(self.W3.T @ V2) - self.th3 # scalar
75         O = np.tanh(b3)
76         return x, V1, V2, O, b1, b2, b3
77
78     # one SGD step (choose  $\mu$  at random outside)
79     def sgd_step(self, x, t):
80         x, V1, V2, O, b1, b2, b3 = self.forward(x)
81
82         # output delta and grads
83         delta3 = (t - O) * (1.0 - np.tanh(b3)**2) # scalar
84         dTheta3 = delta3
85         dW3 = delta3 * V2.T # (1 x M2) logically; we store as (M2
x 1)
86         # convert to our column storage:
87         dW3_col = (delta3 * V2) # (M2 x 1)
88
89         # hidden 2
90         delta2 = (self.W3 * delta3) * self.gprime(b2) # (M2 x 1)
91         dTheta2 = delta2
92         dW2 = delta2 @ V1.T # (M2 x M1)
93
94         # hidden 1
95         delta1 = (self.W2.T @ delta2) * self.gprime(b1) # (M1 x 1)
96         dTheta1 = delta1
97         dW1 = delta1 @ x.T # (M1 x 2)
98
99         # updates: weights plus, thresholds minus
100         self.W1 += self.lr * dW1
101         self.W2 += self.lr * dW2
102         self.W3 += self.lr * dW3_col
103
104         self.th1 -= self.lr * dTheta1
```

```
105         self.th2 -= self.lr * dTheta2
106         self.th3 -= self.lr * dTheta3
107
108     def predict_outputs(self, X):
109         """Return raw outputs (tanh before sign) for a whole set X."""
110         out = np.empty((X.shape[0], 1), dtype=float)
111         for i in range(X.shape[0]):
112             _, _, _, 0, *_ = self.forward(X[i])
113             out[i, 0] = 0
114         return out
115
116     def save_csv(self, prefix=""):
117         np.savetxt(prefix + "w1.csv", self.W1, delimiter=",", fmt="%.6f")
118         np.savetxt(prefix + "w2.csv", self.W2, delimiter=",", fmt="%.6f")
119         np.savetxt(prefix + "w3.csv", self.W3, delimiter=",", fmt="%.6f") # M2 x 1
120         np.savetxt(prefix + "t1.csv", self.th1, delimiter=",", fmt="%.6f")
121         np.savetxt(prefix + "t2.csv", self.th2, delimiter=",", fmt="%.6f")
122         np.savetxt(prefix + "t3.csv", np.array([[self.th3]]), delimiter=",", fmt="%.6f")
123         print("Saved w1,w2,w3,t1,t2,t3")
124
125 # ----- training script -----
126
127 def main():
128     # load data
129     train = try_load_csv("training_set")
130     val = try_load_csv("validation_set")
131     Xtr, Ttr = train[:, :2], train[:, 2]
132     Xva, Tva = val[:, :2], val[:, 2]
133
134     # map {0,1} -> {-1,+1} if needed
135     if set(np.unique(Ttr)) == {0.0, 1.0}:
136         Ttr = 2*Ttr - 1
137         Tva = 2*Tva - 1
138         print("Mapped targets {0,1} -> {-1,+1}")
139
140     n = Xtr.shape[0]
141     m = Xva.shape[0]
142     print(f"train {n} val {m}")
143
144     # hyperparameters
145     lr = 0.01
146     M1, M2 = 10, 5
147     max_epochs = 1000
148
149     model = TwoLayerPerceptron(M1, M2, lr=lr, seed=0)
150
151     rng = np.random.RandomState(123)
152     for epoch in range(1, max_epochs+1):
153         # one epoch = n random pattern updates
154         for _ in range(n):
155             mu = rng.randint(0, n) # like randi(n)
156             x = Xtr[mu]
157             t = float(Ttr[mu])
158             model.sgd_step(x, t)
```

```
159
160     # validation error after each epoch
161     outputs = model.predict_outputs(Xva)    # (m,1)
162     Cval = classification_error_from_outputs(outputs, Tva)
163     print(f"Epoch {epoch:4d} | Validation error: {Cval:.3f}")
164
165     if Cval < 0.12:
166         print("Reached target < 12% on validation.")
167         break
168
169     model.save_csv()
170     return model
171
172 if __name__ == "__main__":
173     np.random.seed(0)
174     main()
175
```